

Cautarea binara

1. Ce este cautarea binara

Cautarea binara este un algoritm eficient pentru gasirea unui element intr-un vector sortat. Algoritmul elimina la fiecare pas jumatate din intervalul de cautare.

2. Conditie esentiala

Vectorul trebuie sa fie sortat inainte de aplicarea cautarii binare. Daca vectorul nu este sortat, rezultatul este incorect.

3. Ideea algoritmului

Se mentine un interval [st, dr]. La fiecare pas se calculeaza mijlocul m. Daca a[m] este egal cu valoarea cautata, algoritmul se opreste. Daca valoarea cautata este mai mica, se continua in stanga, altfel in dreapta.

4. Exemplu explicat pas cu pas

a = [1, 3, 5, 7, 9, 11], cautam x = 7
Pas 1: st=0, dr=5 -> m=2, a[m]=5 < 7 -> cautam in dreapta
Pas 2: st=3, dr=5 -> m=4, a[m]=9 > 7 -> cautam in stanga
Pas 3: st=3, dr=3 -> m=3, a[m]=7 -> gasit

5. Implementare C++ (iterativ)

```
#include <iostream>
using namespace std;

int main(){
    int n; cin >> n;
    int a[100000];
    for(int i=0;i<n;i++) cin >> a[i];
    int x; cin >> x;

    int st = 0, dr = n-1;
    int poz = -1;
    while(st <= dr){
        int m = (st + dr) / 2;
        if(a[m] == x){ poz = m; break; }
        if(a[m] < x) st = m + 1;
        else dr = m - 1;
    }

    cout << poz;
    return 0;
}
```

6. Ce returneaza algoritmul

Daca elementul este gasit, se returneaza pozitia lui. Daca nu este gasit, se returneaza -1 (sau se afiseaza un mesaj corespunzator).

7. Variante uzuale

- verificarea existentei unui element - prima aparitie / ultima aparitie - cea mai mare valoare $\leq x$ - cea mai mica valoare $\geq x$

8. Complexitate

La fiecare pas se elimina jumatate din interval. Complexitate timp: $O(\log n)$ Memorie suplimentara: $O(1)$

9. Greseli frecvente

- aplicarea pe vector nesortat - calcul gresit al mijlocului - bucla infinita (st si dr nu se modifica corect) - confuzia intre cautare binara si cautare liniara

10. Checklist de BAC

- verifici ca vectorul este sortat - stii schema st-dr-m - intelegi fiecare pas din exemplu - explici de ce este $O(\log n)$